COLLEGE OF COMPUTER STUDIES

CMSC 203 – Object-Oriented Programming

Final Project: Object-Oriented Application Development

EVENT REGISTRATION SYSTEM

A Web-Based, Object-Oriented Management System

Submitted by:

[Group member 1]
[Group member 2]
[Group member 3]

Submitted to:

[Instructor name]

October 2026

Table of Contents

1	Introduction
2	Statement of the Problem
3	Objectives
4	Scope and Limitations
5	System Features
6	System Design (OOP, UML Class Diagram, Flowcharts)
7	Screenshots of the Application
8	Testing Results
9	Conclusion
10	References

1. Introduction

Schools and colleges hold many events every year, such as seminars, workshops, meet-ups and department nights. Organizers usually collect sign-ups through paper forms, group chats or spreadsheets. These methods make it hard to know how many seats are left, who actually registered and who showed up.

The Event Registration System is a web-based application built for the College of Computer Studies. Students create an account, browse upcoming events and reserve spots. An administrator creates and manages events, views every registration and records attendance. The system was written in Python using the Flask framework and applies the object-oriented programming concepts taught in CMSC 203: classes and objects, constructors, encapsulation, inheritance, polymorphism, abstraction, method overloading, abstract classes and custom exception handling. The web interface was approved by the instructor as the graphical user interface for this project.

2. Statement of the Problem

Manual event registration causes the following problems:

- Organizers cannot easily tell how many spots remain, so events can be overbooked.
- The same student can sign up several times, and records are scattered across forms and chats.
- Attendance is not recorded in a consistent way, so there is no reliable history of who attended.
- Event details (date, time, place, capacity) change often and are hard to keep updated for everyone.
- Student and event data can be lost or corrupted when stored in unorganized files.

This project addresses these problems with one system that keeps events, accounts, registrations and attendance in one place and enforces the registration rules automatically.

3. Objectives

General objective: to design and develop an object-oriented Event Registration System that lets students reserve event spots online and lets an administrator manage events and attendance.

Specific objectives:

1. Provide participant sign up and login, and a separate administrator login.
2. Allow the administrator to add, edit, delete, search and display events.
3. Allow participants to browse, search and filter events, reserve spots, and cancel reservations.
4. Prevent duplicate reservations and prevent reservations that exceed an event's capacity.
5. Allow the administrator to view participants and registrations and to record attendance.
6. Save and load all data in a JSON file without losing records.
7. Apply the OOP principles of encapsulation, inheritance, polymorphism and abstraction in a modular code base.
8. Handle invalid input, empty fields, wrong data types and file errors, including at least one custom exception.

4. Scope and Limitations

Scope

- Two kinds of users: participants (students) and one administrator.
- Free events only; a reservation reserves one or more spots (no payment).
- Event management, registration, cancellation, attendance tracking, search, filtering and dashboards.
- Data stored in a local JSON file (data.json) with atomic saving and automatic upgrade of older records.
- Runs in any modern web browser; the server is started with Python.

Limitations

- There is only one administrator account, configured through environment variables.
- The JSON file suits a small school deployment; it is not meant for thousands of simultaneous users. The Repository class makes it possible to replace it with a real database later.
- There is no email notification, password reset or QR-code check-in.
- A participant's reservation is for the whole event; seats are not individually assigned.
- Participants cannot edit their own profile after sign up.

5. System Features

5.1 Functional requirements

Requirement	Implementation
User login	Participant sign up / login (passwords stored as salted hashes); separate administrator login
Add records	Admin adds events; participants add registrations (reserve spots)
Edit records	Admin edits event details; capacity cannot go below already reserved spots
Delete records	Admin deletes events (blocked if registrations exist); participants cancel their reservations
Search records	Search boxes for events, participants, registrations and attendance; category filter for events
Display records	Dashboards, event cards, tables, profile and attended-events list
Save data	JsonRepository.save() writes data.json atomically (temporary file, then replace)
Load data	JsonRepository.load() reads data.json and upgrades older records automatically

5.2 Participant features

- Dashboard with totals, featured events and recent registrations
- Browse events, search, filter by category

- Reserve spots (1 or more) for a free event; duplicate and over-capacity reservations are rejected
- View and cancel reservations (not after attendance has been recorded)
- Profile page and Attended Events history

5.3 Administrator features

- Dashboard with total events, participants, registrations and attendees
- Add, edit, delete and search events (with title, category, date, time, location, capacity, description)
- View participants and all registrations with search
- Attendance management: mark each registration as attended or absent

5.4 Validation and error handling

- Empty fields, invalid email, short password, duplicate email or Student ID
- Invalid dates and times, past event dates, capacity that is not a whole number or is less than 1
- Permission checks: participants cannot use admin pages and vice versa
- Damaged or unreadable data file reports a clear error instead of crashing
- A failed request never saves half-finished changes (transaction rollback)

6. System Design

6.1 Architecture

The code is split into layers so that every module has one job:

Layer	Module	Responsibility
Presentation	templates/, static/	HTML, CSS and JavaScript user interface (browser)
Routes	routes.py	Thin Flask routes; convert HTTP requests to service calls and results to JSON
Services	services/	Business rules: AuthService, EventService, RegistrationService
Models	models/	Domain classes: Entity, Person, Participant, Admin, Event, Registration, Database
Data access	repository.py	Repository (abstract) and JsonRepository
Support	exceptions.py, validation.py, config.py	Custom exceptions, reusable validators, settings

6.2 Object-oriented concepts applied

Concept	Where it is used
Classes and objects	Participant, Admin, Event, Registration, Database, the three services and JsonRepository
Constructors	__init__ of every class, e.g. Event(title, date, time, location, capacity, ...); factories Event.create(), Participant.create(), Registration.for_participant()
Encapsulation	Private attributes (_capacity, _password_hash, _attendance_status, _quantity) exposed through validated properties and methods; callers cannot store invalid values
Inheritance	Entity → Person → Participant / Admin; Entity → Event; Entity → Registration; Repository → JsonRepository; EventSystemError → ValidationError, ConflictError → DuplicateAccountError, etc.
Polymorphism	describe(), to_dict(), from_dict() and check_password() behave differently in each subclass but are called the same way
Abstraction	Abstract classes Entity, Person and Repository (ABC with @abstractmethod) define contracts; services depend on Repository, not on the JSON file
Method overloading	EventService.get() accepts an event id (str) or an Event object using functools.singledispatchmethod; many

	methods use default arguments
Interfaces / abstract classes	Entity, Person and Repository cannot be instantiated (tested in test_repository_is_abstract)
Custom exceptions	EventSystemError hierarchy: ValidationError, AuthenticationError, PermissionDeniedError, NotFoundError, ConflictError, DuplicateAccountError, DuplicateRegistrationError, EventFullError, StorageError

6.3 UML Class Diagram

Figure 1 shows the classes, their attributes and operations, inheritance (hollow triangles), aggregation (hollow diamonds) and dependencies (dashed arrows). Italic names are abstract. The same diagram is available as docs/uml_class_diagram.png and .svg.

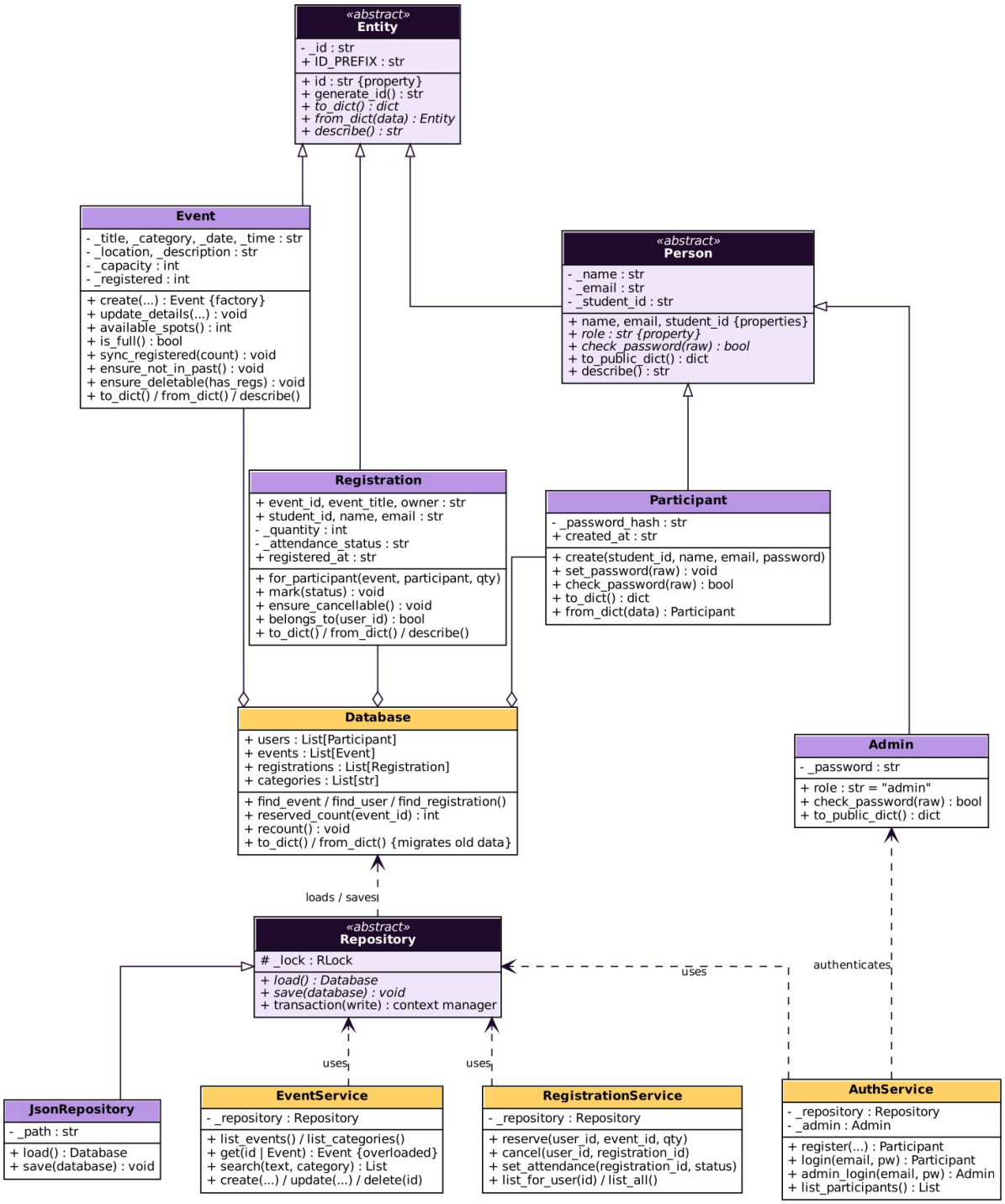


Figure 1. UML Class Diagram

6.4 Flowcharts

Figure 2 shows how a user moves through the system. Figure 3 follows one request in detail: reserving a spot.

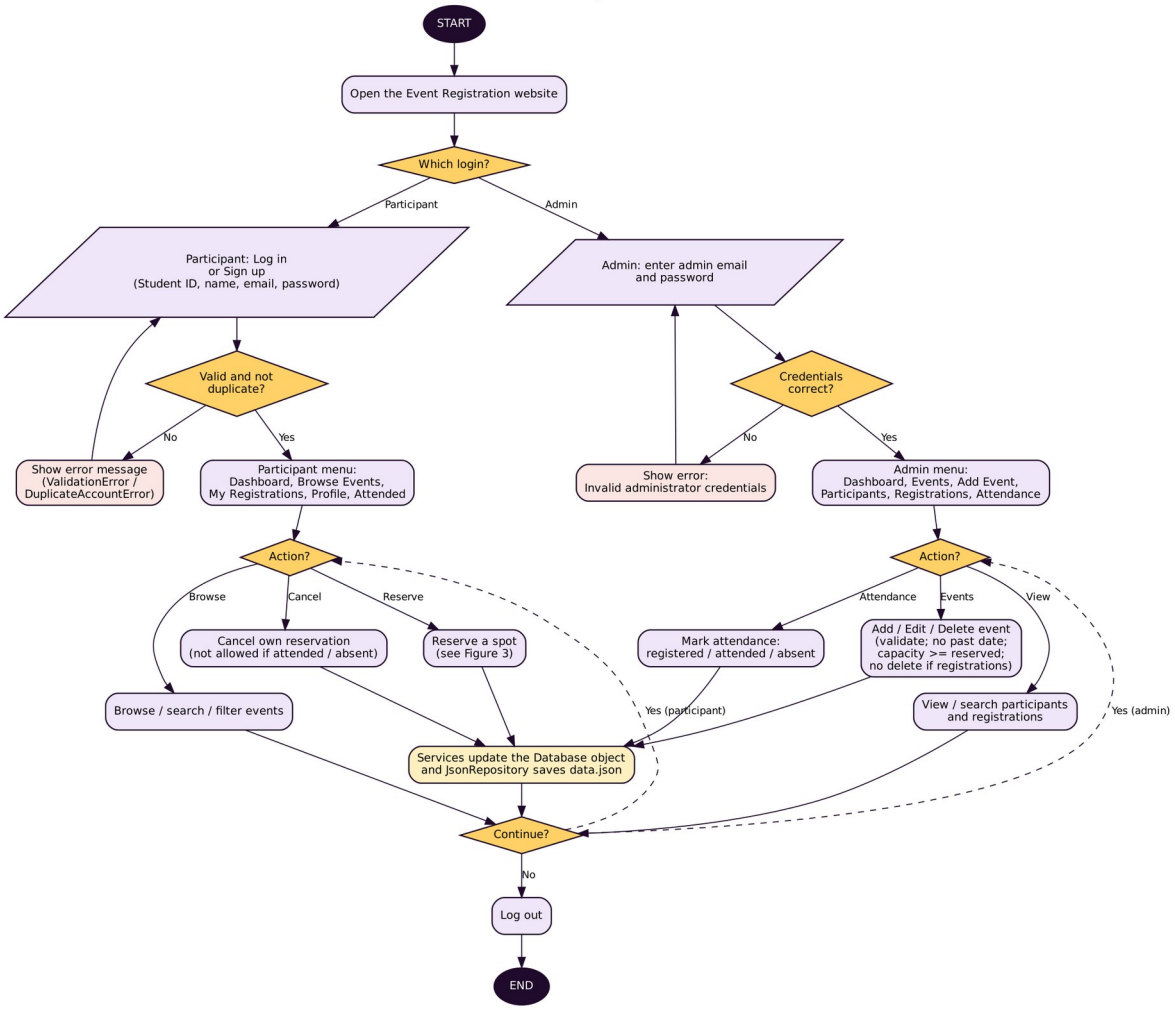


Figure 2. Overall system flowchart

Figure 3. Reservation Process Flowchart (RegistrationService.reserve)

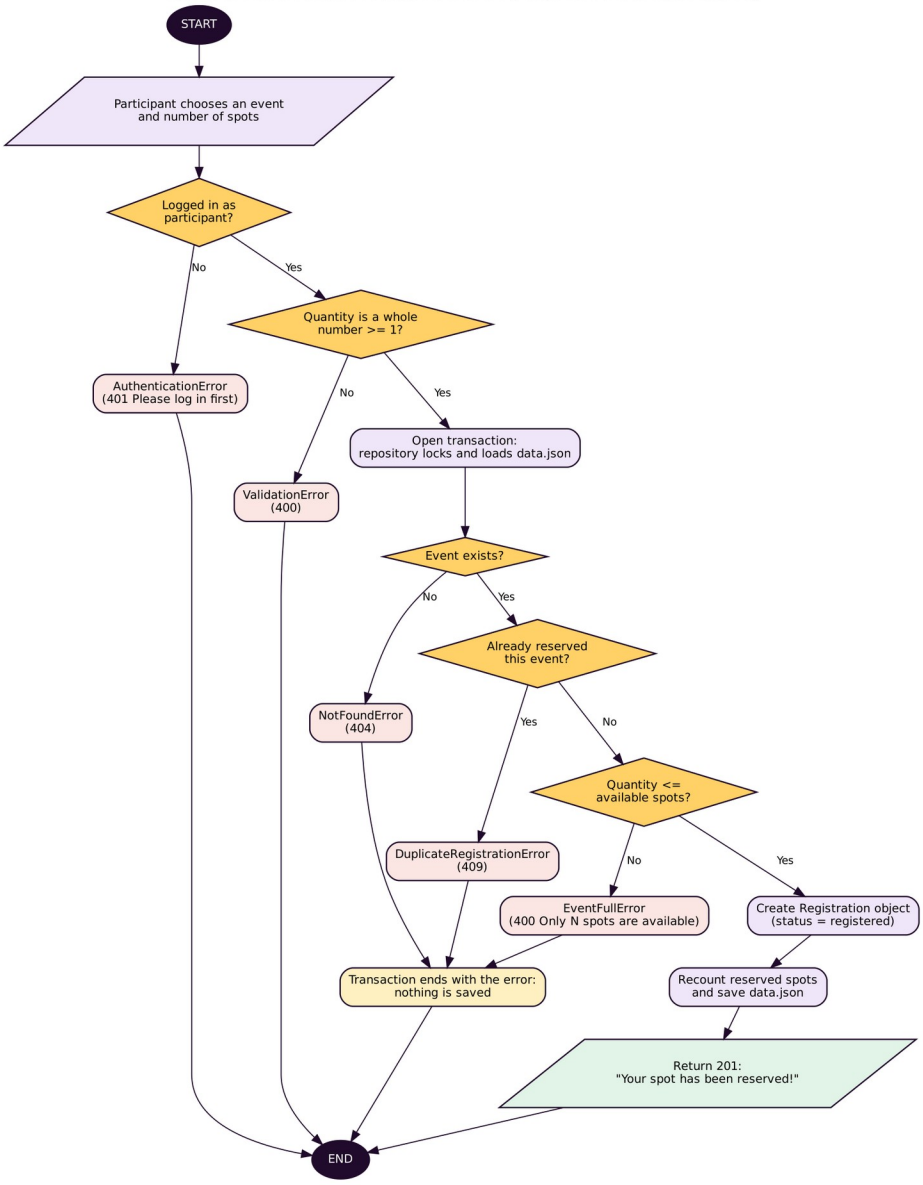


Figure 3. Reservation process flowchart

6.5 Data storage

All data is stored in data.json with four collections: events, categories, registrations and users. The Database class holds them as objects while the program runs. Every change happens inside Repository.transaction(), which locks the file, loads the data, lets the service make its changes, recalculates the reserved spots, and saves. If an exception is raised, nothing is saved.

7. Screenshots of the Application

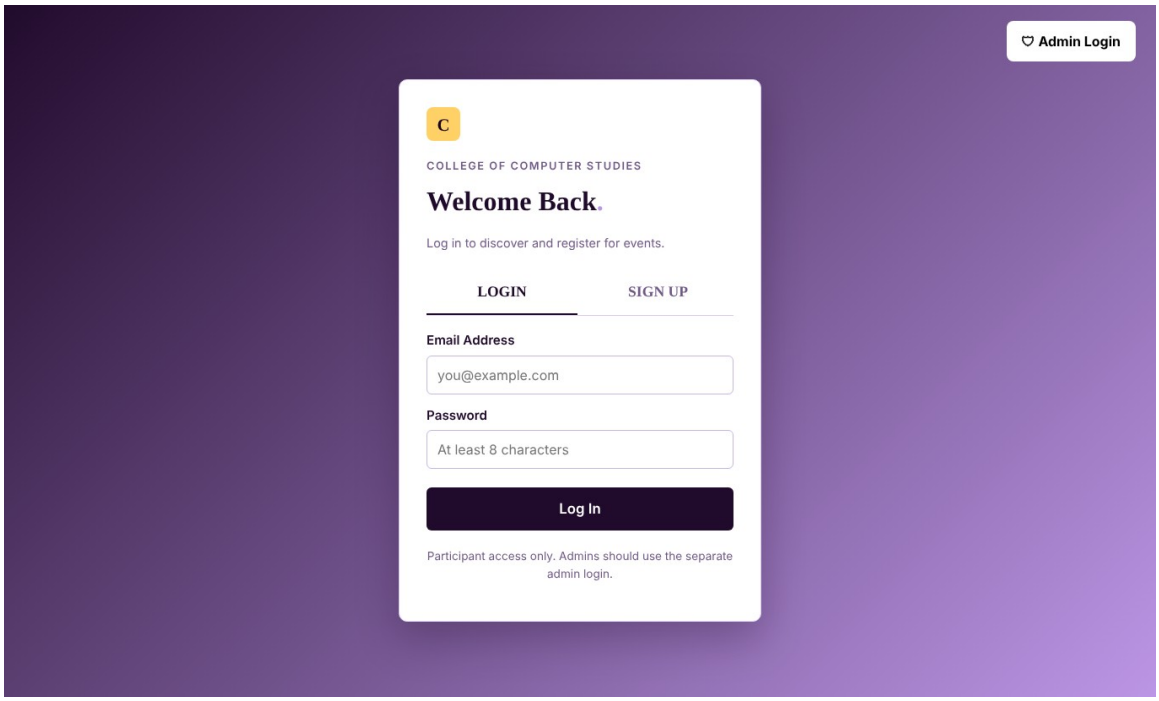


Figure 4. Login and sign-up screen

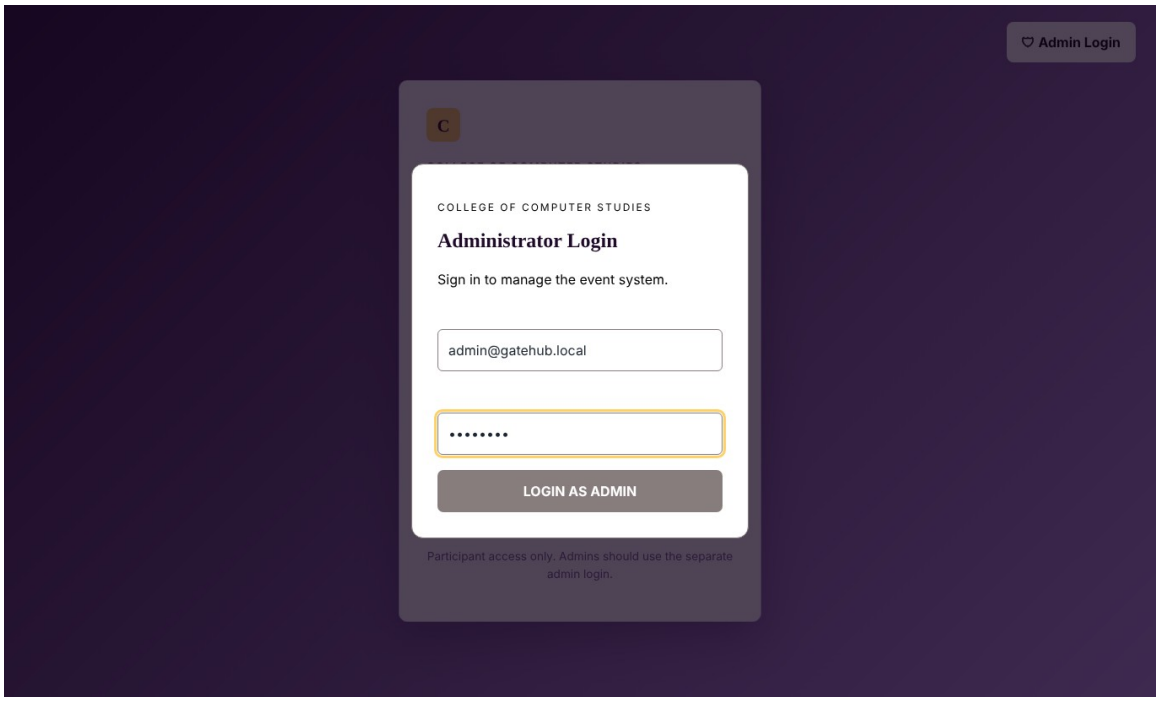


Figure 5. Administrator login window

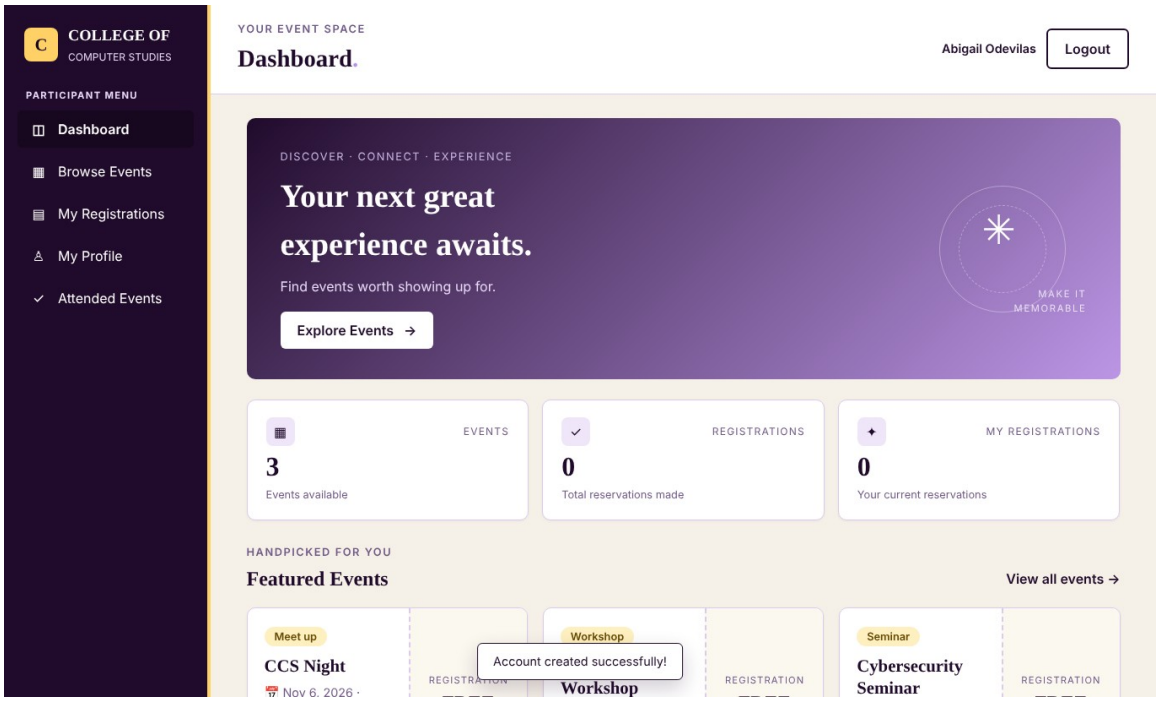


Figure 6. Participant dashboard

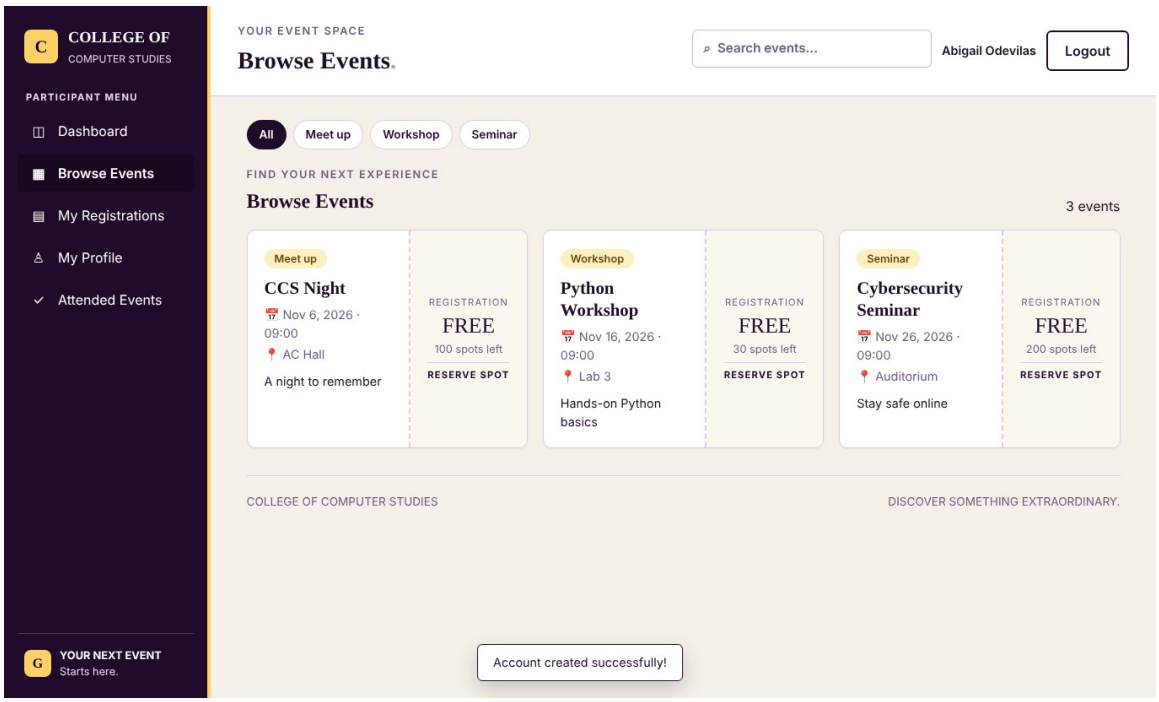


Figure 7. Browse events (search and category filter)

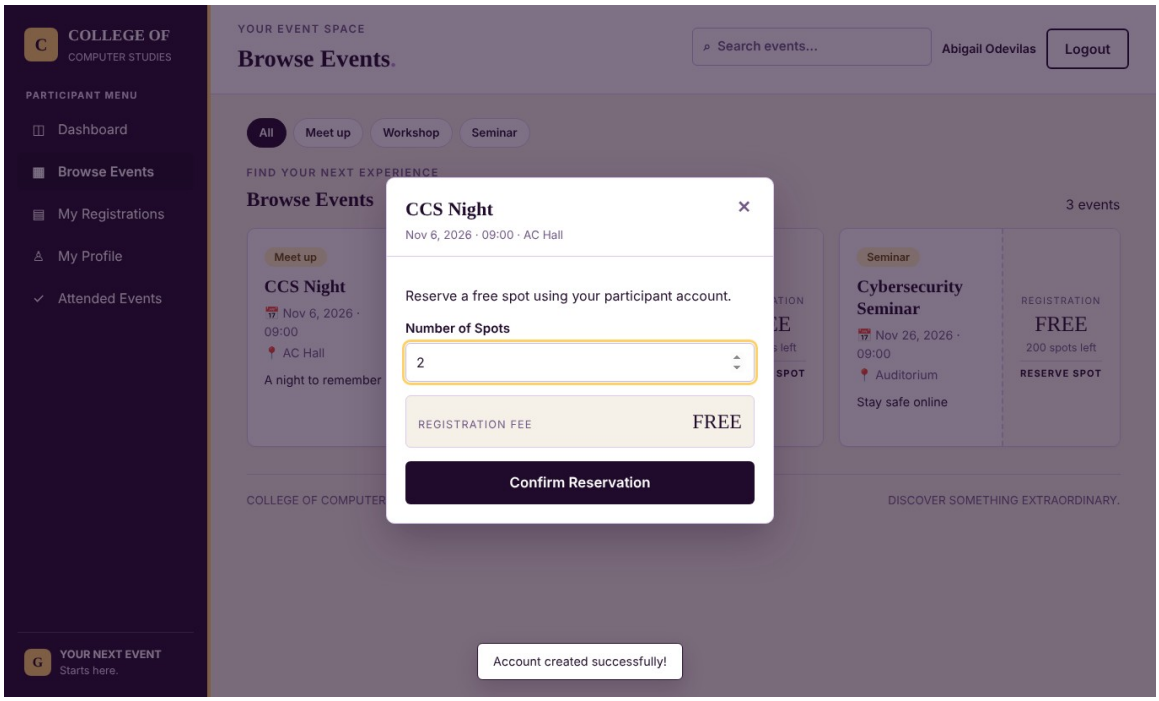


Figure 8. Reserve spots window

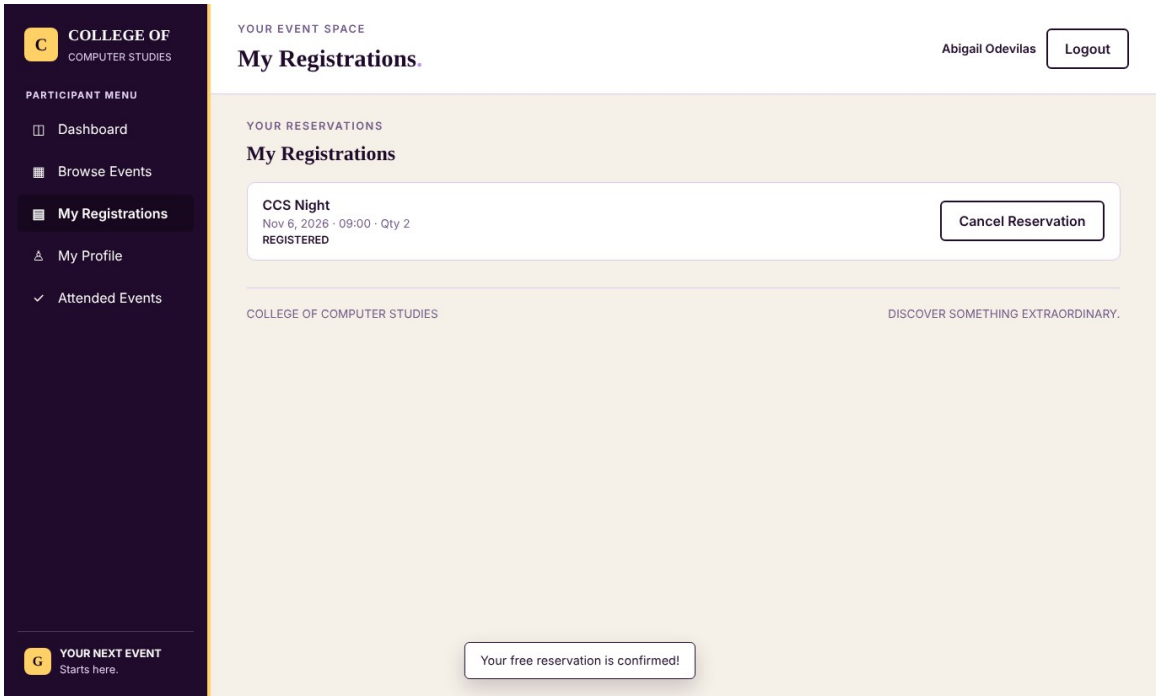


Figure 9. My Registrations

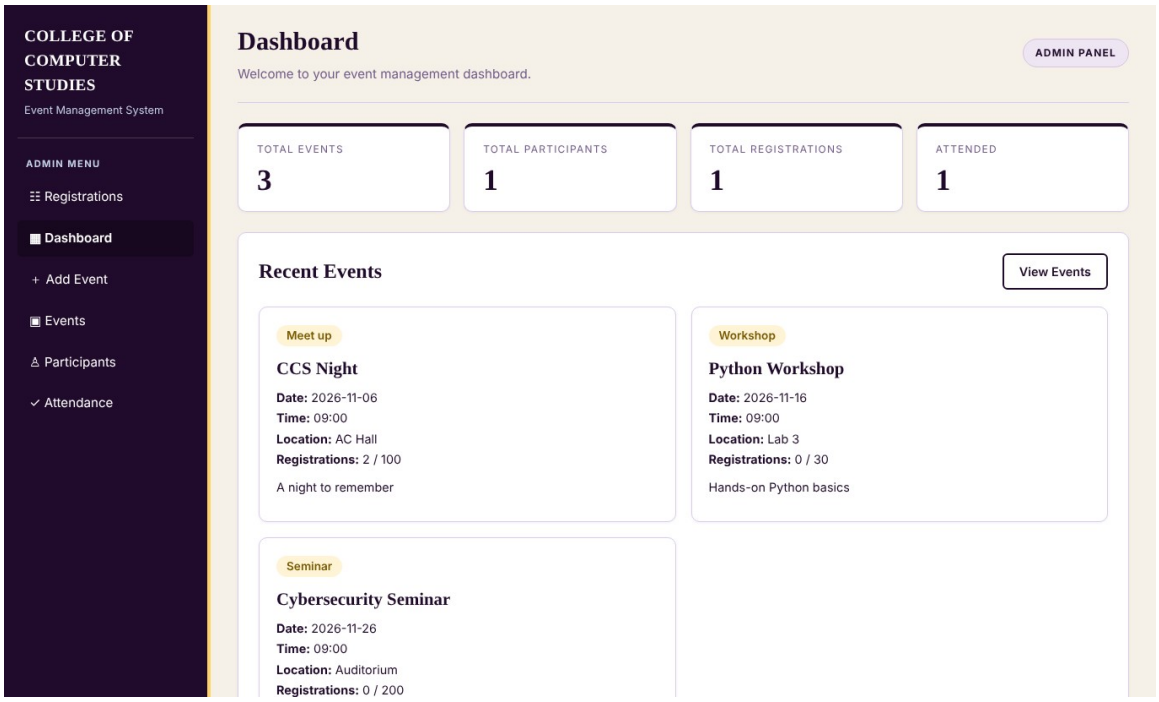


Figure 10. Admin dashboard

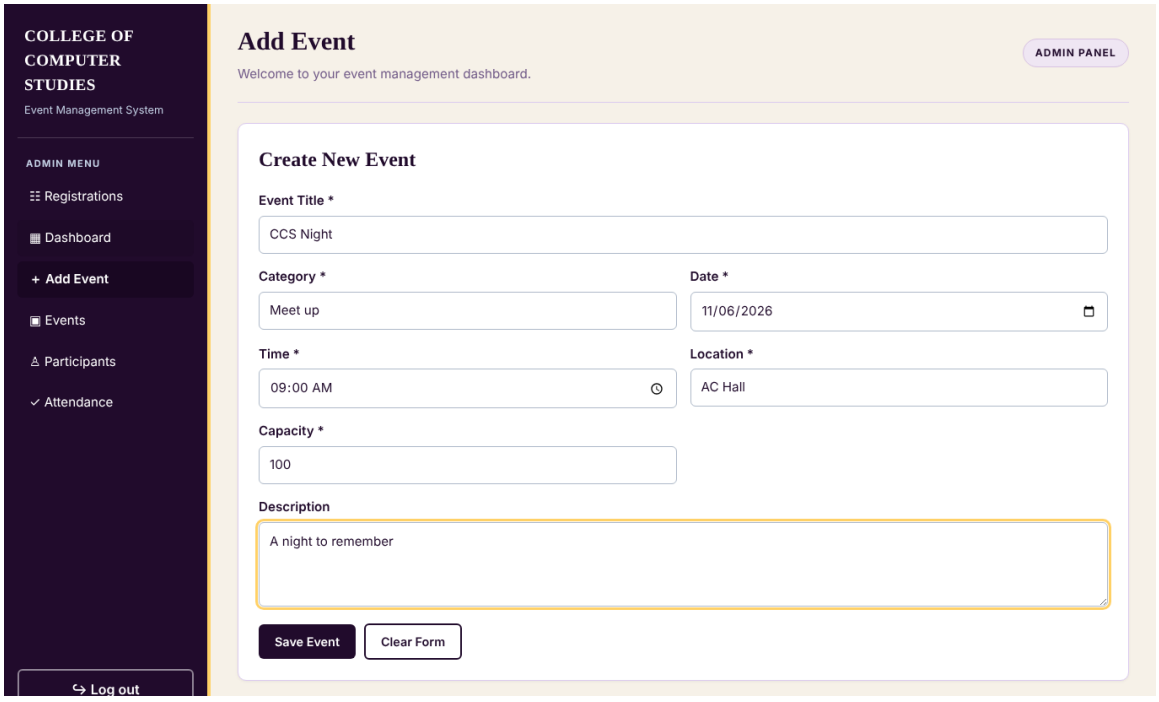


Figure 11. Admin: add event form

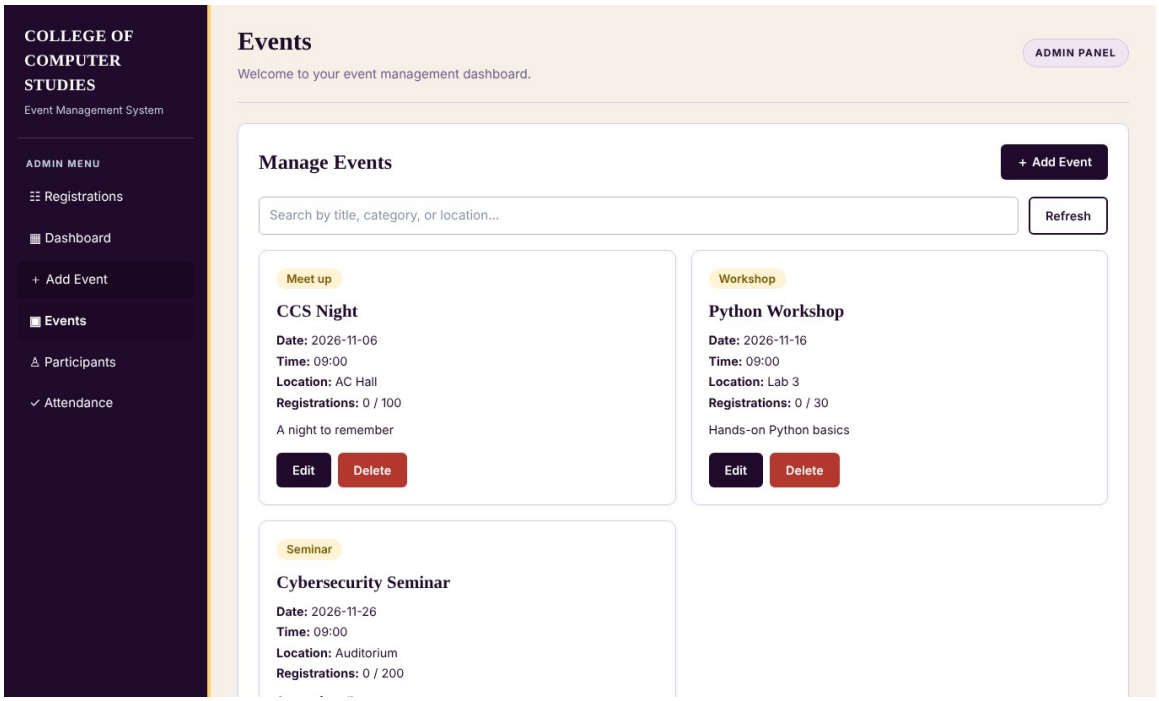


Figure 12. Admin: manage events

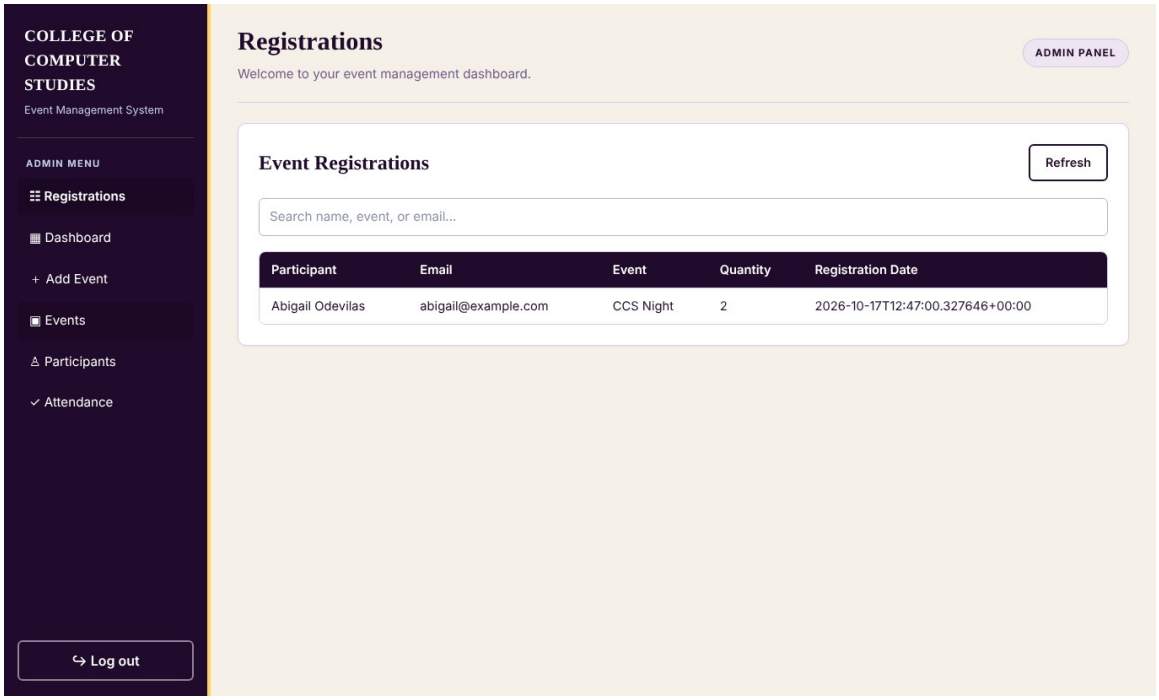


Figure 13. Admin: event registrations

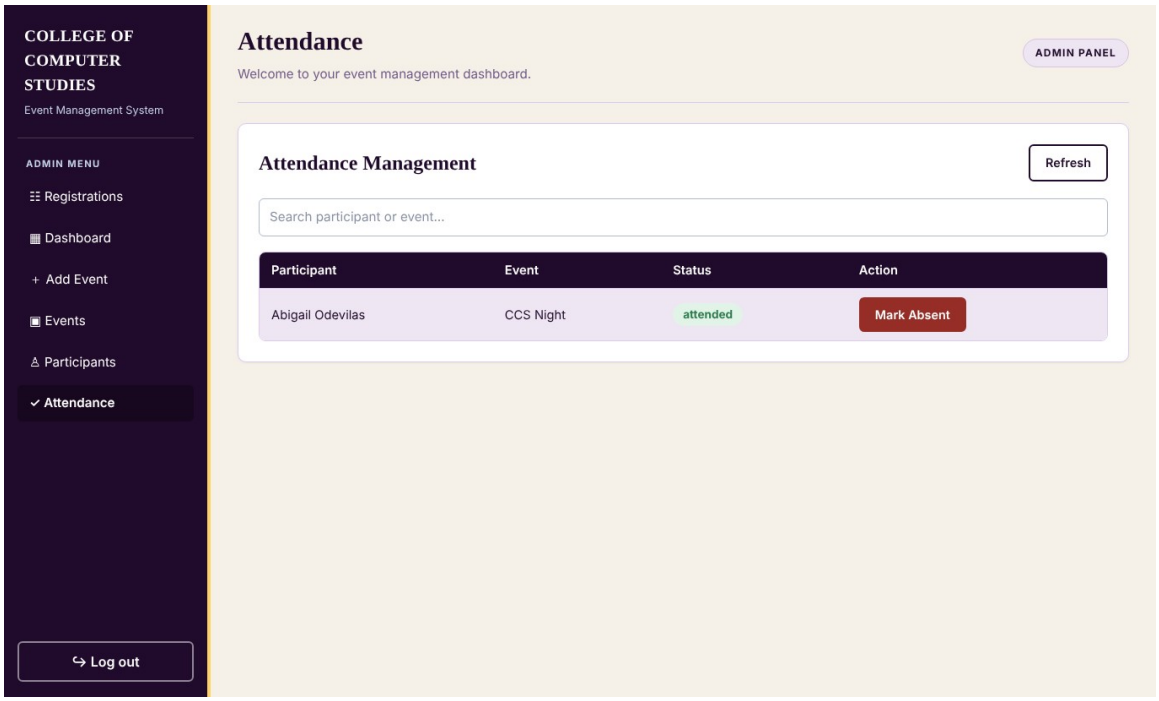


Figure 14. Admin: attendance management

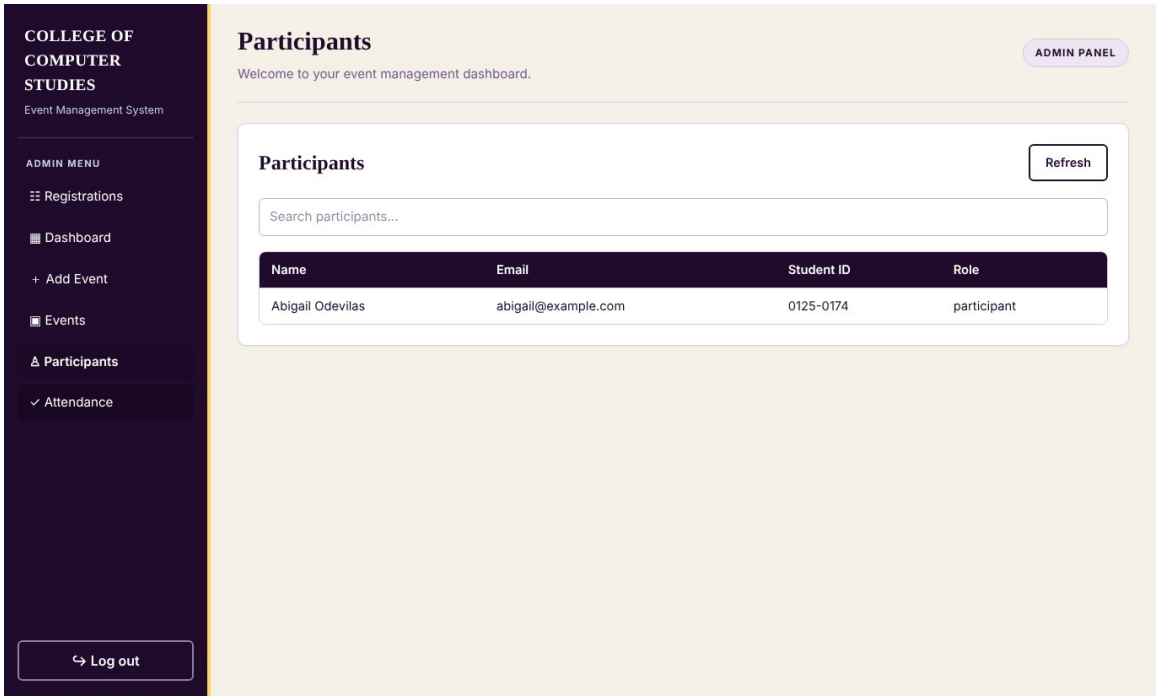


Figure 15. Admin: participants

8. Testing Results

8.1 Automated tests

25 automated unit and integration tests (tests/test_system.py, run with "python -m unittest discover -s tests -v") were executed. Result: Ran 25 tests – OK (all passed).

#	Test	Area	Result
1	admin login and permissions	Accounts and login	Passed
2	duplicates rejected	Accounts and login	Passed
3	password hash never exposed	Accounts and login	Passed
4	signup login logout	Accounts and login	Passed
5	signup validation	Accounts and login	Passed
6	wrong password and admin email	Accounts and login	Passed
7	cannot lower capacity or delete with registrations	Event management	Passed
8	create edit delete and categories	Event management	Passed
9	event input errors	Event management	Passed
10	custom exceptions	Custom exceptions	Passed
11	event rules	Model classes	Passed
12	participant encapsulation and hashing	Model classes	Passed
13	polymorphism describe	Model classes	Passed
14	registration status and cancel rules	Model classes	Passed
15	repository is abstract	Model classes	Passed
16	attendance flow	Reservations and attendance	Passed
17	cancel frees spots	Reservations and attendance	Passed
18	cannot cancel someone elses	Reservations and attendance	Passed
19	duplicate and overbooking blocked	Reservations and attendance	Passed
20	invalid quantity	Reservations and attendance	Passed
21	reserve updates count	Reservations and attendance	Passed
22	damaged file raises storage error	Data file	Passed
23	failed transaction does not save	Data file	Passed
24	legacy data is migrated	Data file	Passed
25	missing file gives empty database	Data file	Passed

8.2 Manual test cases (through the web interface)

These scenarios were verified through the automated API tests above and a full walkthrough of the running application in a browser (sign up, admin login, add event, reserve, view registrations, mark attendance), during which no JavaScript errors were reported.

No.	Test case	Expected result	Actual result
1	Sign up with all fields valid	Account created, participant dashboard opens	Passed
2	Sign up with an email already used	Error "An account with this email already exists."	Passed
3	Log in with a wrong password	Error "Invalid email or password."	Passed
4	Admin logs in with correct credentials	Admin dashboard opens	Passed
5	Admin adds an event with all fields	Event appears in Events list and in the participant's Browse Events	Passed
6	Admin adds an event with an empty title	Form is rejected with an error message	Passed
7	Admin adds an event with a past date	Error "The event date cannot be in the past."	Passed
8	Participant reserves 2 spots	Reservation saved; available spots reduced by 2	Passed
9	Participant reserves the same event again	Error "You have already reserved this event."	Passed
10	Another participant asks for more spots than remain	Error "Only N spots are available."	Passed
11	Participant cancels a reservation	Reservation removed; spots returned to the event	Passed
12	Admin marks a registration as attended	Status changes; shows in the participant's Attended Events	Passed
13	Admin deletes an event that has registrations	Deletion refused with an explanation	Passed
14	Open the admin pages as a participant	Access denied (403)	Passed
15	Start the program with a damaged data.json	Clear storage error instead of a crash	Passed

9. Conclusion

The Event Registration System meets its objectives. Students can register for events and see their history, and administrators can manage events and attendance, all through a clean web interface. The back end is written with object-oriented design: abstract base classes define contracts, subclasses

specialise behaviour, attributes are encapsulated behind validated properties, and a custom exception hierarchy turns every kind of error into a clear message. The layered structure (routes, services, models, repository) keeps the code readable and testable, as shown by the 25 passing automated tests.

Possible future improvements are email confirmations, QR-code check-in, a waiting list for full events, multiple administrators with roles, and moving from the JSON file to a database such as SQLite or PostgreSQL by writing a new Repository subclass.

10. References

9. Python Software Foundation. (2026). Python 3 documentation. https://docs.python.org/3/
10. Python Software Foundation. (2026). abc – Abstract Base Classes; functools.singledispatchmethod. https://docs.python.org/3/library/
11. Pallets Projects. (2026). Flask documentation. https://flask.palletsprojects.com/
12. Pallets Projects. (2026). Werkzeug documentation: Security helpers. https://werkzeug.palletsprojects.com/
13. Object Management Group. (2017). Unified Modeling Language (UML) specification, version 2.5.1. https://www.omg.org/spec/UML/
14. Graphviz. (2026). Graphviz documentation (used to draw the diagrams). https://graphviz.org/documentation/
15. CMSC 203 – Object-Oriented Programming course materials and Final Project instructions.